

React Interview Prep

Topic 8 — useRef: Complete Guide

Analogy · DOM Access · Mutable Storage · Prev Value · forwardRef ·
useImperativeHandle · Rules · Q&A;

- Covers All examples from conversation — nothing skipped
- Series React Interview Prep — Session 8

by Akshay Chavhan • React Interview Series

1. Like You're 5 — The Secret Notebook

You have a secret notebook: you can write anything in it, nobody knows when you write in it (no alarms), and it's always there between page refreshes.

■ *useRef = a secret box that holds a value **WITHOUT** telling React about it. Change it → no re-render. It remembers the value across renders silently.*

2. What is useRef?

useRef returns a mutable object that persists across renders. Changing it never triggers a re-render — React doesn't know or care.

```
const ref = useRef(initialValue);  
// ref looks like this:  
{ current: initialValue }  
// always accessed via .current
```

■ *useRef has TWO use cases: 1. Access real DOM elements directly 2. Store mutable values without triggering re-render*

3. Use Case 1 — DOM Access

Attach ref to any JSX element — React sets .current to the actual DOM node after render.



```
function InputFocus() {  
  const inputRef = useRef(null); // null before mount  
  function handleClick() {  
    inputRef.current.focus(); // real DOM method ■  
    inputRef.current.scrollIntoView(); // scroll to element ■  
    inputRef.current.value; // read input value ■  
    inputRef.current.getBoundingClientRect(); // dimensions ■  
  }  
  return (  
    <>  
    <input ref={inputRef} type='text' />  
    <button onClick={handleClick}>Focus Input</button>  
    </>  
  );  
}
```

4. Use Case 2 — Mutable Storage Without Re-render

Store values that change but don't need to update the UI — timer IDs, previous values, flags.

```
function Timer() {  
  const [count, setCount] = useState(0);  
  const timerRef = useRef(null); // stores timer ID silently  
  function start() {  
    timerRef.current = setInterval(() => {
```

```
setCount(c => c + 1);
}, 1000);
}
function stop() {
clearInterval(timerRef.current); // access stored ID ■
}
}
```

■ *If you stored timerRef in useState, it would trigger a re-render on every start/stop. useRef stores it silently — zero re-renders.*

5. useRef vs useState

	useState	useRef
Triggers re-render	■ Yes	■ No
Persists across renders	■ Yes	■ Yes
Access	value directly	.current
Update is synchronous	■ No (batched)	■ Yes (immediate)
Use when	UI must reflect change	Silent storage / DOM access
Show in UI	■ Yes	■ Avoid — UI won't update

■ Wrong

```
// ■ ref for UI value
const count = useRef(0);
count.current += 1;
// UI never updates!
return <p>{count.current}</p>;
```

■ Correct

```
// ■ state for UI value
const [count, setCount]
= useState(0);
setCount(c => c + 1);
return <p>{count}</p>; // ■
```

6. Tracking Previous State Value

A classic useRef pattern — store the previous render's value.

```
function App() {
  const [name, setName] = useState('Akshay');
  const prevName = useRef(name); // store previous
  useEffect(() => {
    prevName.current = name; // updates AFTER render
  }, [name]);
  return (
    <div>
      <p>Current: {name}</p>
      <p>Previous: {prevName.current}</p>
      <input onChange={e => setName(e.target.value)} />
    </div>
  );
}
```

■ **Effect runs AFTER render** — *prevName.current holds old value during current render, then updates to new value after. Perfect previous-value tracker.*

7. Tracking If Component Is Mounted

```
function App() {
  const isMounted = useRef(false);
  useEffect(() => {
    isMounted.current = true; // mounted
    return () => { isMounted.current = false; }; // unmounted
  });
}
```

```
}, []);  
async function fetchData() {  
  const data = await fetch('/api/data').then(r => r.json());  
  // ■ Only update state if still mounted  
  if (isMounted.current) { setData(data); }  
  // Prevents 'setState on unmounted component' warning  
}  
}
```

8. forwardRef — Pass Ref to Child Component

By default, ref doesn't work on custom components. forwardRef solves this.

■ Wrong

```
// ■ ref on custom component
// ref.current = null always!
function Parent() {
  const ref = useRef(null);
  return <CustomInput ref={ref}/>;
  // ref is null – won't work ■
}
```

■ Correct

```
// ■ wrap child in forwardRef
const CustomInput = forwardRef(
  (props, ref) => (
    <input ref={ref} {...props}/>
  )
  // ref.current = DOM input ■
)
```

■ forwardRef passes ref as second argument to child, which forwards it to the real DOM node.

9. useImperativeHandle — Control What Ref Exposes

Limit what the parent can do with the child's ref — expose only specific methods.

```
const CustomInput = forwardRef((props, ref) => {
  const inputRef = useRef(null);
  useImperativeHandle(ref, () => ({
    focus: () => inputRef.current.focus(), // ■ allowed
    clear: () => { inputRef.current.value = ''; } // ■ allowed
  }));
  // parent CANNOT access .style, .id etc ■
  return <input ref={inputRef} {...props} />;
});

// Parent usage:
ref.current.focus(); // ■ allowed
ref.current.clear(); // ■ allowed
ref.current.style; // ■ undefined – not exposed
```

■ useImperativeHandle gives child control over its own API — parent only calls what child allows.

10. Real World — Auto Scroll (Chat App)

```
function ChatBox({ messages }) {
  const bottomRef = useRef(null);
  useEffect(() => {
    // Auto scroll to bottom when new message arrives
    bottomRef.current?.scrollIntoView({ behavior: 'smooth' });
  }, [messages]);
  return (
    <div className='chat-box'>
      {messages.map(msg => <p key={msg.id}>{msg.text}</p>)}
      <div ref={bottomRef} /> { /* invisible anchor at bottom */ }
    </div>
  );
}
```


11. Key Rules of useRef

Rule	Detail
Never read/write ref.current during render	Side effect during render = impure component. Do it in useEffect or event handlers only.
Never use ref for values shown in UI	Ref changes don't trigger re-render — UI stays stale. Use useState instead.
ref.current is always synchronous	Unlike batched state updates, ref.current = value is immediate and always fresh.
ref on custom component = null by default	Must use forwardRef to pass ref through a custom component to a DOM node.
Initial value only used on first render	useRef(initialValue) — the value is only used on mount, like useState.

12. Interview Q&A;

Q

Beginner

Q1. What is useRef and what are its two main uses?

→ useRef returns a mutable object { current: value } that persists across renders. First use: accessing real DOM nodes directly by attaching ref to JSX elements. Second use: storing mutable values like timer IDs or flags without triggering re-renders.

Q

Beginner

Q2. What is the difference between useRef and useState?

→ useState triggers a re-render when updated and the new value is available on the next render. useRef never triggers re-renders, updates synchronously, and is accessed via .current. Use useState when UI must reflect the change. Use useRef for silent storage or DOM access.

Q

Intermediate

Q3. Why can't you pass ref to a custom component directly?

→ React doesn't forward the ref prop to custom components by default — it's a reserved prop. You need to wrap the child in forwardRef, which receives the ref as a second argument and forwards it to the actual DOM node inside.

Q

Intermediate

Q4. What is forwardRef?

→ forwardRef is a React API that lets a parent component pass a ref through a custom child to a real DOM node inside it. Without forwardRef, ref on a custom component is always null.

Q

Intermediate

Q5. Can you store a previous state value using useRef?

→ Yes — store the value in `useEffect` that runs when state changes. The effect runs after render, so the ref holds the previous value during the current render and updates to the new value after. Classic pattern for tracking previous props or state.

Q

Expert

Q6. What is `useImperativeHandle` and when would you use it?

→ `useImperativeHandle` works with `forwardRef` to control what the parent can access via ref. Instead of exposing the entire DOM node, you expose only specific methods — giving the child control over its own API. Use it in reusable component libraries to prevent misuse of internal DOM.

Q

Expert

Q7. Why should you not read or write `ref.current` during render?

→ Refs are mutable and changes aren't tracked by React. Reading or writing during render makes the component impure — same inputs could produce different outputs. In Strict Mode, React renders twice, which would cause double mutation bugs. Always use refs in `useEffect` or event handlers.